String (Internal Working of String) and String Builder

1. String Class in Java

What is a String?

A String in Java is a sequence of characters. It is a class in Java that represents text data. Strings are objects of the String class.

```java
String name = "Java Programming";
String message = new String("Hello World");
```

String Immutability

Strings in Java are **immutable**, which means once a String object is created, it cannot be changed. Any operation that appears to modify a string actually creates a new String object.

```java
String str = "Hello";
str = str + " World";  // Creates a new String object
System.out.println(str); // Output: Hello World
```

2. Internal Working of String

String Pool (String Constant Pool)

The String Pool is a special memory area in the heap where Java stores String literals. When you create a String using double quotes, Java first checks if the same string exists in the pool.

```java
String str1 = "Java";        // Stored in String Pool
String str2 = "Java";        // Points to same object in pool
String str3 = new String("Java"); // Creates new object in heap

System.out.println(str1 == str2);  // true (same reference)
System.out.println(str1 == str3);  // false (different references)
System.out.println(str1.equals(str3)); // true (same content)
```

Memory Management

- **String literals**: Stored in String Pool
- **String objects**: Created using `new` keyword, stored in heap memory
- **Garbage Collection**: Unused String objects are eligible for garbage collection

```java
String a = "Hello";     // String Pool
String b = "Hello";     // Points to same object in pool
String c = new String("Hello"); // Heap memory
```

3. String Methods

Common String Methods

```java
String text = "Java Programming";

// Length
int length = text.length(); // 16

// Character at index
char ch = text.charAt(0); // 'J'

// Substring
String sub = text.substring(5); // "Programming"
String sub2 = text.substring(0, 4); // "Java"

// Index methods
int index = text.indexOf('P'); // 5
int lastIndex = text.lastIndexOf('g'); // 15

// Case conversion
String upper = text.toUpperCase(); // "JAVA PROGRAMMING"
String lower = text.toLowerCase(); // "java programming"

// Replace
String replaced = text.replace('a', 'A'); // "JAvA Programming"

// Split
String[] words = text.split(" "); // ["Java", "Programming"]

// Trim
String spaced = "  Hello  ";
String trimmed = spaced.trim(); // "Hello"
```

4. StringBuilder Class

Why StringBuilder?

Since Strings are immutable, concatenating multiple strings creates many temporary objects, leading to performance issues. StringBuilder provides a mutable sequence of characters.

```java
// Inefficient with String
String result = "";
for(int i = 0; i < 1000; i++) {
    result += i; // Creates new String object each time
}

// Efficient with StringBuilder
StringBuilder sb = new StringBuilder();
for(int i = 0; i < 1000; i++) {
    sb.append(i); // Modifies existing buffer
}
String result = sb.toString();
```

StringBuilder Methods

```java
StringBuilder sb = new StringBuilder();

// Append
sb.append("Hello");
sb.append(" ");
sb.append("World");
System.out.println(sb); // Hello World

// Insert
sb.insert(5, " Java"); // Hello Java World

// Delete
sb.delete(5, 10); // Hello World

// Replace
sb.replace(0, 5, "Hi"); // Hi World

// Reverse
sb.reverse(); // dlroW iH

// Convert to String
String finalString = sb.toString();
```

StringBuilder Constructors

```java
// Default constructor (capacity 16)
StringBuilder sb1 = new StringBuilder();

// With initial capacity
StringBuilder sb2 = new StringBuilder(50);

// With initial string
StringBuilder sb3 = new StringBuilder("Hello");
```

5. String vs StringBuilder Performance

Memory Usage Comparison

```java
// String concatenation - creates multiple objects
String str = "A";
str += "B"; // Creates new object
str += "C"; // Creates another new object

// StringBuilder - uses resizable buffer
StringBuilder sb = new StringBuilder("A");
sb.append("B"); // Modifies existing buffer
sb.append("C"); // Modifies existing buffer
```

When to Use Each

- **String**: When the text won't change frequently
- **StringBuilder**: When you need to perform many string operations (concatenation, insertion, deletion)

6. StringBuffer vs StringBuilder

Key Differences

- **StringBuilder**: Not thread-safe, faster performance
- **StringBuffer**: Thread-safe (synchronized), slower performance

```java
// StringBuilder - single-threaded applications
StringBuilder sb = new StringBuilder("Hello");

// StringBuffer - multi-threaded applications
StringBuffer sbf = new StringBuffer("Hello");
```

Quiz Questions

1. What happens when you create a String using double quotes in Java? a) It creates a new object in heap memory b) It checks the String pool first and reuses if exists c) It always creates a new object d) It stores in stack memory

Answer: b) It checks the String pool first and reuses if exists

2. Which of the following is true about String immutability? a) Strings can be modified after creation b) String objects are stored in stack memory c) Once created, String content cannot be changed d) String immutability affects only String literals

Answer: c) Once created, String content cannot be changed

3. What is the main advantage of StringBuilder over String concatenation? a) StringBuilder is thread-safe b) StringBuilder provides better performance for multiple concatenations c) StringBuilder uses less memory always d) StringBuilder is easier to use

Answer: b) StringBuilder provides better performance for multiple concatenations

4. What does the following code output?

```java
String a = "Java";
String b = "Java";
System.out.println(a == b);
```

a) false
b) true
c) Compilation error
d) Runtime error

Answer: b) true

5. Which method is used to convert StringBuilder to String? a) getString() b) convertToString() c) toString() d) makeString()

Answer: c) toString()

Practice Coding Questions

1. Write a Java program that takes a string and counts the number of vowels and consonants in it.

2. Create a program using StringBuilder that reverses each word in a sentence while keeping the word order the same. For example: "Hello World Java" should become "olleH dlroW avaJ".

3. Write a method that checks if two strings are anagrams of each other (contain the same characters in different order). Use appropriate string methods.

4. Create a program that removes all duplicate characters from a string using StringBuilder. For example: "programming" should become "progamin".

5. Write a Java program that finds the longest palindromic substring in a given string using string manipulation methods.